Concepts Cheat Sheet

Public Transport Tracker — All Key Concepts Explained

Review this before your promotion walkthrough

Section 1: Architecture Concepts

Microservices Architecture [HLD / Architecture]

Decompose a system into small, independently deployable services

An architecture style where a large application is split into small services, each responsible for a specific business capability. Services communicate via APIs (REST/HTTP or messaging). Each service can be deployed, scaled, and updated independently.

- Each service owns its domain: Transit Data, Route Planning, Alerts, Health
- Services communicate via REST APIs (JSON over HTTP)
- In this project: modular monolith — same JVM, but each service is a separate package/class with its own interface
- Benefit: change Alerts logic without touching Transit Data logic

Remember: In this project we use a modular monolith (not separate deployments), which gives microservice-style boundaries with simpler operations. Mention this trade-off in your review.

High-Level Design (HLD) [HLD / Architecture]

The big picture — what components exist and how they connect

HLD describes the system architecture at a macro level: what services exist, how they communicate, what external systems are involved, and what the data flow looks like. No code-level detail — this is for stakeholders and architects.

- Defines: React SPA → API Gateway → Microservices → Cache → External APIs
- Includes: component diagram, sequence diagrams, data flow, tech stack choices
- HLD answers: What are the boxes? How do they talk to each other?
- Artifacts: architecture diagram, ADRs (Architecture Decision Records)

Remember: HLD is the conversation starter. It lets senior engineers critique the design before a line of code is written.

Low-Level Design (LLD) [LLD / Architecture]

The implementation blueprint — how each component works internally

LLD zooms into each component from HLD and describes its internal structure: class diagrams, method signatures, data structures, and algorithms. This is what developers use to write actual code.

- Defines: class names, interfaces, method signatures, data models
- Includes: TTLCacheService design, AlertsServiceImpl rule engine, package structure
- LLD answers: How is this class structured? What algorithm is used?
- Artifacts: class diagrams, sequence diagrams with method calls, DB schema (if any)

Remember: LLD is what you defend in a code review. Every design choice here should be justifiable.

12-Factor App [Architecture / Ops]

12 rules for building production-ready cloud-native applications

A methodology for building SaaS/cloud apps that are portable, scalable, and maintainable. Key factors most relevant to this project:

- Factor 3 — Config: Store config (API keys, TTL values) in environment variables, never hardcoded
- Factor 4 — Backing Services: Treat external transit APIs as attached resources (replaceable)
- Factor 6 — Processes: App is stateless — cache lives in memory, not sticky sessions
- Factor 7 — Port binding: App exports service via port 8080 (self-contained)
- Factor 11 — Logs: Treat logs as event streams (structured JSON to stdout)
- Factor 12 — Admin: Health/readiness via Spring Actuator

Remember: When asked 'is this production-ready?', cite 12-Factor compliance. It's the gold standard.

Section 2: Design Patterns

SOLID Principles [Design Patterns / OOP]

Five principles for writing maintainable object-oriented code

A set of five design principles that, when followed, produce code that is easy to change, extend, and test:

- S — Single Responsibility: One class, one reason to change. TransitDataService only fetches transit data.
- O — Open/Closed: Open for extension, closed for modification. Add a new API by implementing TransitDataService interface.
- L — Liskov Substitution: Subtypes must be substitutable for their base type. Mock implementations in tests.
- I — Interface Segregation: Don't force clients to depend on unused methods. Separate interfaces for read vs write.
- D — Dependency Inversion: Depend on abstractions. Controllers depend on TransitDataService interface, not Impl.

Remember: In your review: point to a specific class and explain which SOLID principle it demonstrates and why.

Strategy Pattern [Design Patterns / GoF]

Swap algorithms at runtime without changing the client

Define a family of algorithms, put each in a separate class, make them interchangeable. The client selects the strategy to use. In this project, cache eviction policy is a strategy.

- CacheEvictionStrategy interface with implementations: TTLEviction, LRUEviction
- TTLCacheService is configured with a strategy at startup
- Benefit: can change eviction logic without modifying cache code
- Also used: TransitClientFactory selects the correct API client strategy per city

Remember: Strategy = plug-in algorithm. If you find yourself writing big if/else blocks selecting algorithms, use Strategy.

Decorator Pattern [Design Patterns / GoF]

Add behaviour to an object without changing its class

Wrap an object with another object that adds functionality. Both implement the same interface. The wrapper delegates to the wrapped object and adds before/after logic.

- CachedTransitDataService decorates TransitDataServiceImpl
- On call: check cache first → if hit, return; if miss, call real service → cache result
- Client (controller) sees only the TransitDataService interface — unaware of caching
- Benefit: add caching/logging/security without polluting the core service class

Remember: Decorator vs Inheritance: Decorator adds behaviour at runtime; inheritance bakes it in at compile time. Prefer Decorator for cross-cutting concerns.

Factory Pattern [Design Patterns / GoF]

Centralize object creation — callers don't need to know concrete types

A class responsible for creating objects of a family, based on some input. The caller just says 'give me an object for NYC' and gets the right API client.

- TransitClientFactory.getClient(city) returns MtaApiClient for NYC, TflApiClient for London, etc.
- New city support = new client class + one line in factory. No changes to callers.
- Pairs well with Open/Closed Principle
- Also: DTO mapping factories convert raw API responses to clean domain DTOs

Remember: Factory is about decoupling creation from usage. When you see 'new ConcreteClass()' scattered across your code, it's Factory time.

Circuit Breaker Pattern [Design Patterns / Resilience]

Stop calling a failing service and return a fallback immediately

Tracks failure rate of an external dependency. When failures exceed a threshold, the circuit 'opens' — subsequent calls immediately return a fallback without hitting the slow/down service. Prevents cascade failures.

- States: CLOSED (normal) → OPEN (failing, use fallback) → HALF-OPEN (test recovery)
- In this project: ApiUnavailableException triggers immediate stale-cache fallback
- Full circuit breaker: Resilience4j library (CircuitBreaker annotation on API client calls)
- Prevents thread starvation when MTA API is slow (3s timeout × 100 threads = bad)

Remember: Circuit Breaker = fail fast + fail gracefully. The difference between '503 Service Unavailable' and 'showing 2-minute-old data' is a circuit breaker.

HATEOAS [API Design]

APIs that tell clients what they can do next — not just data

Hypermedia As The Engine Of Application State. REST API responses include links to related actions and resources, so clients don't need to hard-code URLs. Part of Richardson Maturity Model Level 3.

- Response includes _links: { self: '/api/v1/transit?city=NYC&route=A', alternatives: '/api/v1/routes/plan?...', alerts: '/api/v1/alerts?route=A' }
- Client discovers what's possible from the response, not from documentation
- Makes the API self-documenting and evolvable (change URLs without breaking clients)
- Spring HATEOAS library adds link building helpers

Remember: Interviewers love HATEOAS as a senior-level API design concept. Know the 3 Richardson Maturity levels: 1=Resources, 2=HTTP Verbs, 3=HATEOAS.

Section 3: Resilience & Caching

TTL Cache (Time-To-Live) [Resilience / Performance]

Cache entries expire automatically after a set time

An in-memory store where each entry has an expiry timestamp. After TTL elapses, the entry is considered stale. In a transit app, vehicle positions go stale in 30 seconds — no point showing 5-minute-old bus locations as 'live'.

- Implemented with: ConcurrentHashMap<String, CacheEntry<T>> in Java
- CacheEntry contains: value, insertedAt (Instant), ttlSeconds
- isExpired(): Instant.now().isAfter(insertedAt.plusSeconds(ttlSeconds))
- Stale reads: getStale() returns value even if expired — for fallback on API failure
- Background eviction: ScheduledExecutorService cleans truly dead entries every 60s

Remember: Different data has different TTLs. Vehicle positions: 30s. Route info: 5 minutes. Static stop data: hours. Design TTLs based on data change rate, not on convenience.

Stale-While-Revalidate [Resilience / Caching]

Serve old data immediately, refresh in the background

A caching strategy where: if the cache is stale but the upstream API is unavailable, serve the stale data to the user now while asynchronously trying to refresh it. User gets a response (possibly old) instead of waiting or getting an error.

- Benefit: user always gets a fast response — no waiting for slow APIs
- Implementation: on API timeout, immediately return stale entry with dataAge metadata
- UI shows: 'Data may be 2 minutes old' banner — transparent to user about freshness
- Async refresh: CompletableFuture.runAsync(() -> refreshCache(key)) in background

Remember: Stale-while-revalidate is the difference between a transit app that 'sometimes fails' and one that 'always shows something useful'.

Service Degradation Strategy [Resilience / Architecture]

Gracefully reduce functionality instead of crashing

A planned approach to what the system does when parts fail. Instead of binary up/down, define multiple degradation levels — each worse than the previous, but still usable.

- Level 1 — LIVE: everything working, fresh data
- Level 2 — STALE: API down, serving cached data with age indicator
- Level 3 — DEGRADED: cache miss + API down, return empty list + alert message
- Level 4 — OFFLINE: user-toggled, serve all from cache, no API calls
- Each level has a defined UX response — no surprises for the user

Remember: Interviewers call this 'graceful degradation'. The key phrase: 'We never return 5xx to the client — we always return something meaningful with context about data freshness.'

Section 4: API Design & Web Concepts

REST API Design [API Design]

Resource-oriented, stateless, HTTP-verb-driven API conventions

Representational State Transfer. APIs use HTTP verbs (GET, POST, PUT, DELETE) to operate on resources identified by URLs. Each request is self-contained (stateless).

- GET /api/v1/transit — fetch transit data (idempotent, cacheable)
- GET /api/v1/routes/plan — route planning (query params: origin, destination)
- Response codes: 200 OK, 400 Bad Request (invalid city), 503 converted to 200+stale
- Versioning: /api/v1/ prefix — future versions /api/v2/ don't break existing clients
- Content negotiation: Accept: application/json

Remember: Know the difference between 200 (success), 201 (created), 400 (bad input), 401 (unauth), 403 (forbidden), 404 (not found), 429 (rate limited), 503 (service unavailable).

Server-Sent Events (SSE) [API Design / Real-time]

One-way real-time push from server to browser over HTTP

SSE is an HTTP-based protocol where the server sends a stream of events to the client after the client makes a single GET request. The connection stays open. Simpler than WebSockets for one-way push.

- Use case: alerts stream — server pushes new alerts to React UI without polling
- React client: const es = new EventSource('/api/v1/alerts/stream')
- Spring: return SseEmitter; emitter.send(SseEmitter.event().data(alertDto))
- Advantages over polling: instant delivery, less HTTP overhead, simpler than WS
- Disadvantage: one-direction only (server → client). Use WebSockets if bidirectional needed.

Remember: SSE vs WebSocket: SSE is HTTP, works through proxies, auto-reconnects. WebSocket is a separate protocol, needed for chat/games. For alerts, SSE is the right choice.

OpenAPI / Swagger [API Design / Documentation]

Machine-readable API contract — the source of truth for your API

OpenAPI Specification (formerly Swagger) is a standard format for describing REST APIs. You define endpoints, request/response schemas, and parameters in YAML/JSON. Tools auto-generate documentation and client SDKs.

- Springdoc: add @Operation, @ApiResponse, @Schema annotations to Spring controllers
- Auto-generates: /swagger-ui.html (interactive UI) and /v3/api-docs (JSON spec)
- Contract-first approach: define OpenAPI spec before writing code — aligns team
- Used for: API testing in Postman (import spec), client SDK generation, API governance
- Response schema includes dataAge, alertLevel, cacheSource fields — all documented

Remember: OpenAPI spec is your API contract. If you change an endpoint and don't update the spec, you're breaking the contract. Treat it like code — version control it.

Section 5: Frontend & React Concepts

Single Page Application (SPA) [Frontend / Architecture]

One HTML page, dynamic UI updates via JavaScript — no full page reloads

React applications are SPAs. The browser loads once; all subsequent navigation is handled by JavaScript manipulating the DOM. React Router controls what the user sees based on the URL.

- React 18: Component-based, virtual DOM diffing for efficient updates
- React Router: declarative routing — <Route path='/dashboard' element={<Dashboard/>}>
- State management: React Query for server data (auto-caching, background refresh), Zustand for UI state
- Vite: fast build tool (replaces Create React App) — HMR in dev, optimized bundle in prod
- TypeScript: type safety for DTO interfaces — catches bugs at compile time

Remember: React's strength is component reusability. Every UI element (AlertCard, VehicleMarker, StaleBanner) is a self-contained component with its own props, state, and tests.

Custom Hooks (React) [Frontend / React]

Reusable stateful logic extracted from components

Custom hooks (functions starting with 'use') let you extract and share stateful logic between components without changing the component hierarchy. They make components thin and logic reusable.

- useTransitData(city, route): encapsulates fetch, loading state, error state, and cache
- useAlertStream(): manages EventSource lifecycle, reconnects on drop
- useOfflineMode(): reads/writes localStorage flag, returns [isOffline, toggleOffline]
- Components become thin: const { data, loading, staleAge } = useTransitData('NYC', 'A')
- Testable in isolation: hooks tested with @testing-library/react-hooks

Remember: Custom hooks = the React equivalent of service classes in Spring. They abstract the 'how' so components focus on the 'what to show'.

Section 6: Testing Concepts

TDD — Test Driven Development [Testing]

Write the test first, then write code to make it pass

A development practice where you write a failing test before writing any implementation code. The cycle: Red (failing test) → Green (minimal code to pass) → Refactor (clean up).

- TTLCacheService: write test asserting stale data returned on API failure FIRST
- AlertsService: write test for 'Given delay=20, expect SIGNIFICANT_DELAY alert' FIRST
- Benefits: forces clear API design, prevents over-engineering, built-in regression safety
- Tools: JUnit 5 (@Test, @BeforeEach), Mockito (mock external dependencies)
- Mockito: when(mtaApiClient.fetch(any())).thenThrow(new ApiUnavailableException())

Remember: TDD is a discipline, not just a technique. The discipline is: no production code without a failing test. Even a single line of logic deserves a test.

BDD — Behaviour Driven Development [Testing]

Express tests in plain English — business language, not code

BDD extends TDD by writing test scenarios in a Given-When-Then format that business stakeholders can read. Bridges the gap between technical tests and business requirements.

- Tool: Cucumber with Gherkin syntax
- Example: Feature: Alert generation
- Scenario: Significant delay alert
- Given a route with delay of 20 minutes
- When the alerts service evaluates the route
- Then the alert 'Significant delays - Plan accordingly' is returned
- Benefits: living documentation, stakeholders can verify scenarios, catches misunderstood requirements

Remember: BDD scenarios are requirements you can run as tests. If a stakeholder changes a requirement, you update the Gherkin scenario — the code change follows.

Section 7: CI/CD & Infrastructure

CI/CD Pipeline [DevOps / Infrastructure]

Automate the path from code commit to production deployment

Continuous Integration (CI) = automatically build and test every code change. Continuous Deployment (CD) = automatically deploy passing builds to environments. Eliminates manual, error-prone deployment steps.

- CI trigger: push to any branch → build + test runs automatically
- CD: merge to main → deploy to staging; manual approval → deploy to prod
- Stages: Checkout → Build → Test → Quality Gate → Docker Build → Push → Deploy
- Jenkins: Jenkinsfile defines pipeline as code (checked into the repo)
- GitHub Actions: .github/workflows/ci.yml — runs on every PR

Remember: The Jenkinsfile IS the pipeline. Never configure Jenkins through the UI alone — if the server dies, you lose your pipeline config. Pipeline as Code = version control for your delivery process.

Docker & Containerization [DevOps / Infrastructure]

Package the app + its environment into a portable, reproducible unit

Docker containers bundle the application, its runtime (JDK), and its config into an image that runs identically in dev, staging, and prod. Eliminates 'works on my machine' problems.

- Dockerfile: defines how to build the image — two stages: build (Maven) + run (JDK slim)
- Multi-stage build: final image contains only the JAR, not Maven/source code
- Port: EXPOSE 8080 — container listens on 8080
- Health check: HEALTHCHECK CMD curl -f http://localhost:8080/actuator/health
- Non-root: run as appuser (UID 1001) — security best practice

Remember: Multi-stage builds are a senior-level Docker practice. Mention it: 'The final image is only 150MB because we don't ship Maven or source code in production.'

Spring Boot Actuator / Observability [Ops / Monitoring]

Built-in health, metrics, and management endpoints for production

Spring Boot Actuator adds production-ready features to your Spring app: health checks, metrics, environment info, and more. Essential for Kubernetes liveness/readiness probes and monitoring.

- /actuator/health: is the app up? Returns UP/DOWN. Used by load balancers.
- /actuator/readiness: is the app ready to serve traffic? Checks upstream connections.
- /actuator/metrics: Micrometer metrics — request latency, error rate, JVM heap
- Structured logging: Logback + Logstash JSON encoder → logs parseable by Elasticsearch/Splunk
- SLO example: '99% of requests served in < 500ms' — metrics prove you meet it

Remember: Liveness vs Readiness: Liveness = 'am I still alive?' (restart if not). Readiness = 'am I ready to take requests?' (remove from load balancer if not). Both are important, different.

Quick Reference: Concept → Where Used

Concept	Category	Where in Project
Microservices	HLD / Architecture	4 domain services: Transit, Route, Alerts, Health
12-Factor App	Architecture / Ops	Env vars for config, structured logs, port 8080
SOLID Principles	OOP / Design	SRP in services, DIP in controller→service interface
Strategy Pattern	GoF Pattern	CacheEvictionStrategy, TransitClientFactory
Decorator Pattern	GoF Pattern	CachedTransitDataService wraps real service
Factory Pattern	GoF Pattern	TransitClientFactory creates city-specific API clients
Circuit Breaker	Resilience Pattern	ApiUnavailableException → stale cache fallback
HATEOAS	API Design	_links in TransitResponseDTO for discoverability
TTL Cache	Resilience / Perf	ConcurrentHashMap in TTLCacheService, per-type TTLs
Stale-While-Revalidate	Caching Strategy	getStale() + dataAge in response DTO
Graceful Degradation	Resilience	4-level degradation: LIVE→STALE→DEGRADED→OFFLINE
REST API Design	API Design	GET /api/v1/transit + versioned URLs
Server-Sent Events	Real-time API	GET /api/v1/alerts/stream for alert push
OpenAPI / Swagger	Documentation	/swagger-ui.html via Springdoc library
React SPA	Frontend	Vite+React 18+TypeScript, React Router
Custom Hooks	React / Frontend	useTransitData, useAlertStream, useOfflineMode
TDD	Testing	JUnit 5 + Mockito, test-first for cache + alert logic
BDD	Testing	Cucumber + Gherkin for alert rule scenarios
CI/CD Pipeline	DevOps	GitHub Actions + Jenkins, pipeline as code
Docker	Infrastructure	Multi-stage Dockerfile, port 8080, non-root user
Spring Actuator	Observability	/health, /readiness, /metrics endpoints